

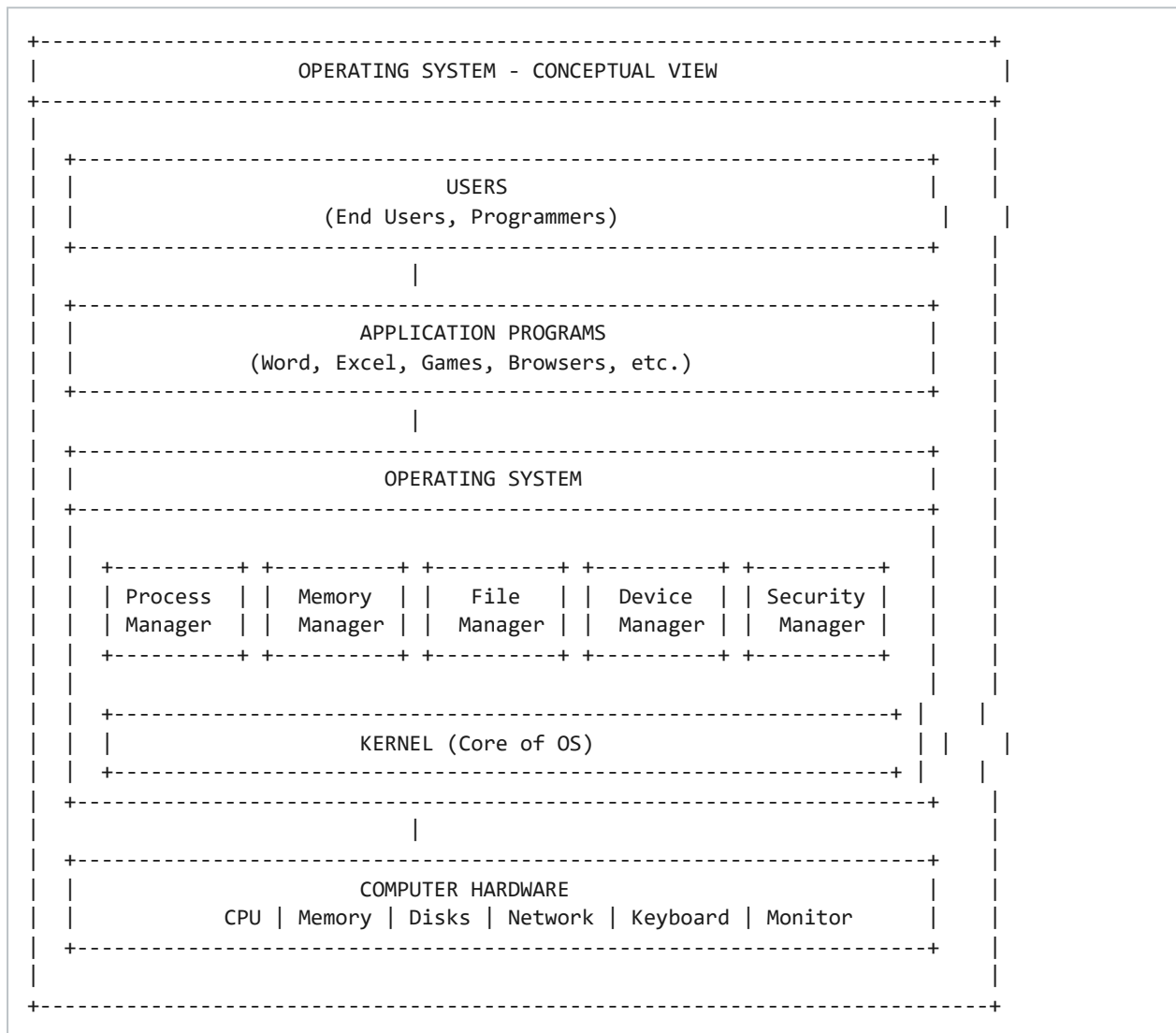
OPERATING SYSTEMS

Unit-1: Introduction to Operating System

Comprehensive Study Notes

1. What is an Operating System?

An **Operating System (OS)** is a **system software** that acts as an **interface** between the **user** and the **computer hardware**. It manages all hardware resources and provides services to application programs.



Functions of Operating System

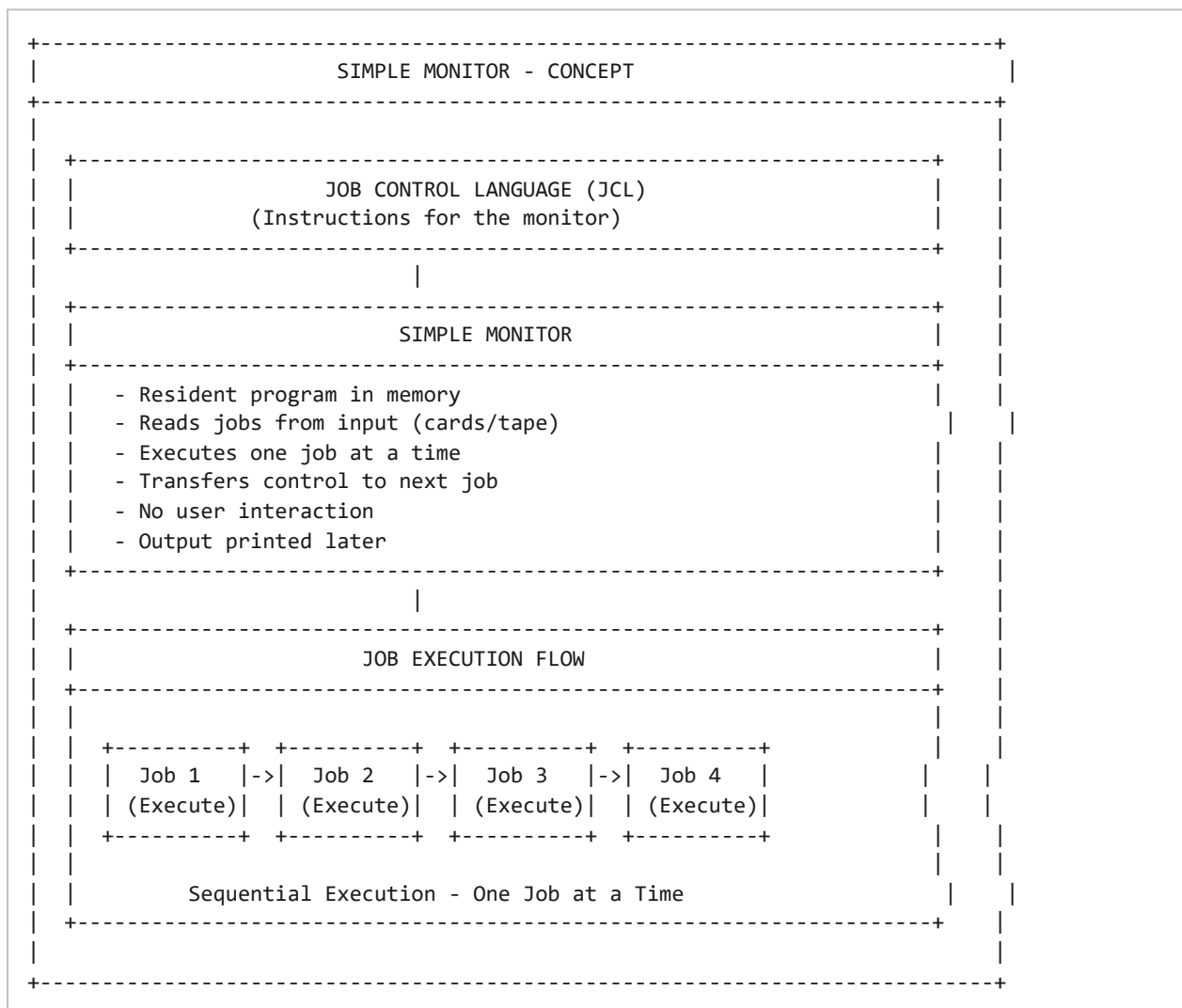
Function	Description
Process Management	Creating, scheduling, and terminating processes
Memory Management	Allocating and deallocating memory to processes

File Management	Creating, deleting, and organizing files
Device Management	Managing I/O devices and device drivers
Security & Protection	Protecting data and resources from unauthorized access
User Interface	Providing interaction between user and system
Error Detection	Detecting and handling errors
Resource Allocation	Allocating resources to different processes

2. Types of Operating Systems

2.1 Simple Monitor (Batch Monitor)

A **Simple Monitor** (also called **Batch Monitor**) was the **earliest type** of operating system. It was a small program that controlled the execution of programs in a **batch** (group) without any user interaction.



Advantages and Disadvantages

Advantages

Simple to implement

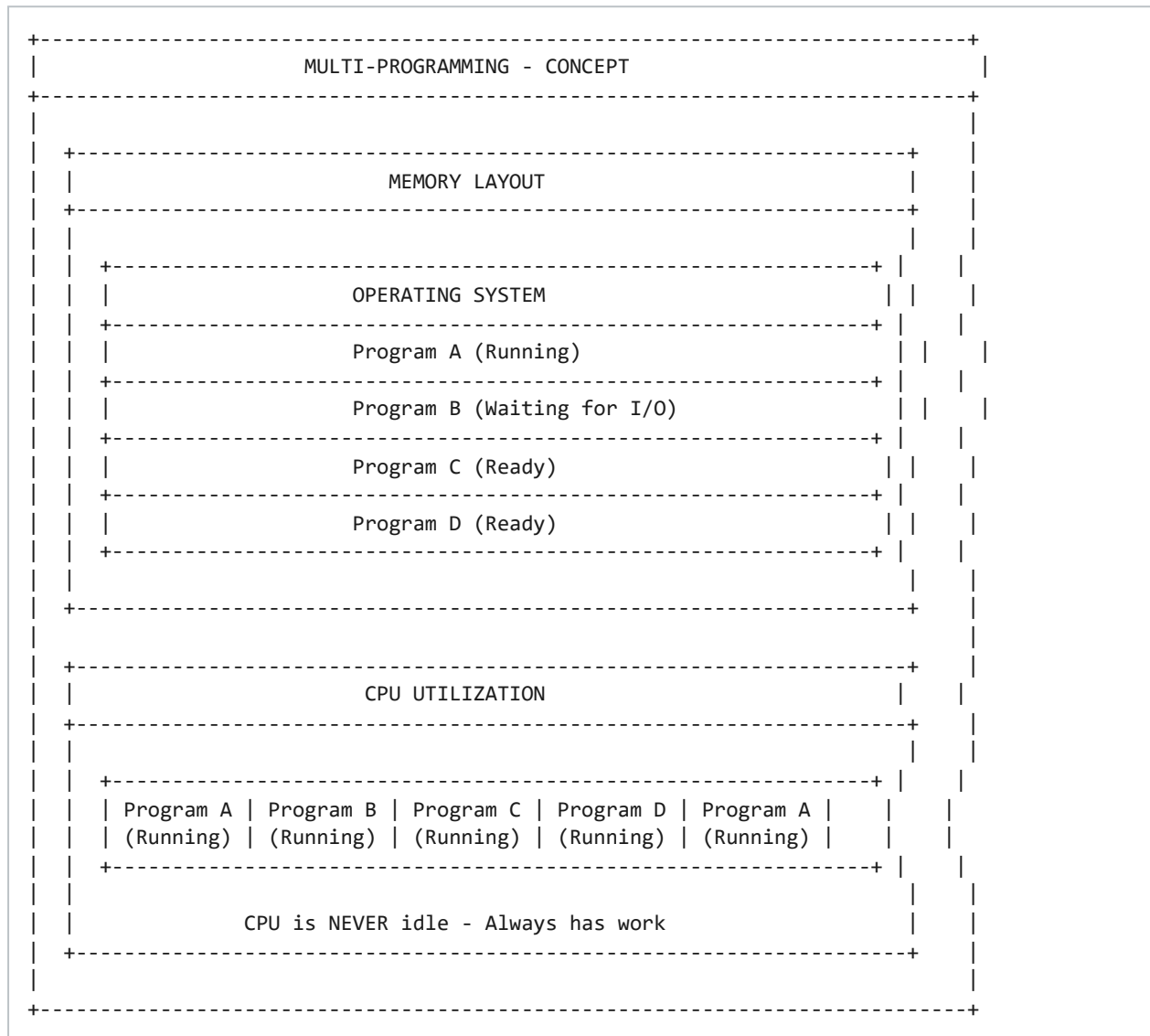
Disadvantages

No user interaction

No CPU idle time (while processing jobs)	CPU idle during I/O operations
Easy to manage	Inefficient for interactive tasks
Reduced overhead	Long turnaround time

1.2 Multi-Programming

Multi-programming (also called **Multitasking**) allows multiple programs to be **loaded into memory simultaneously**. While one program is waiting for I/O, the CPU switches to another program, keeping the CPU **busy** at all times.



Advantages and Disadvantages

Advantages

Maximum CPU utilization

Reduced total execution time

Disadvantages

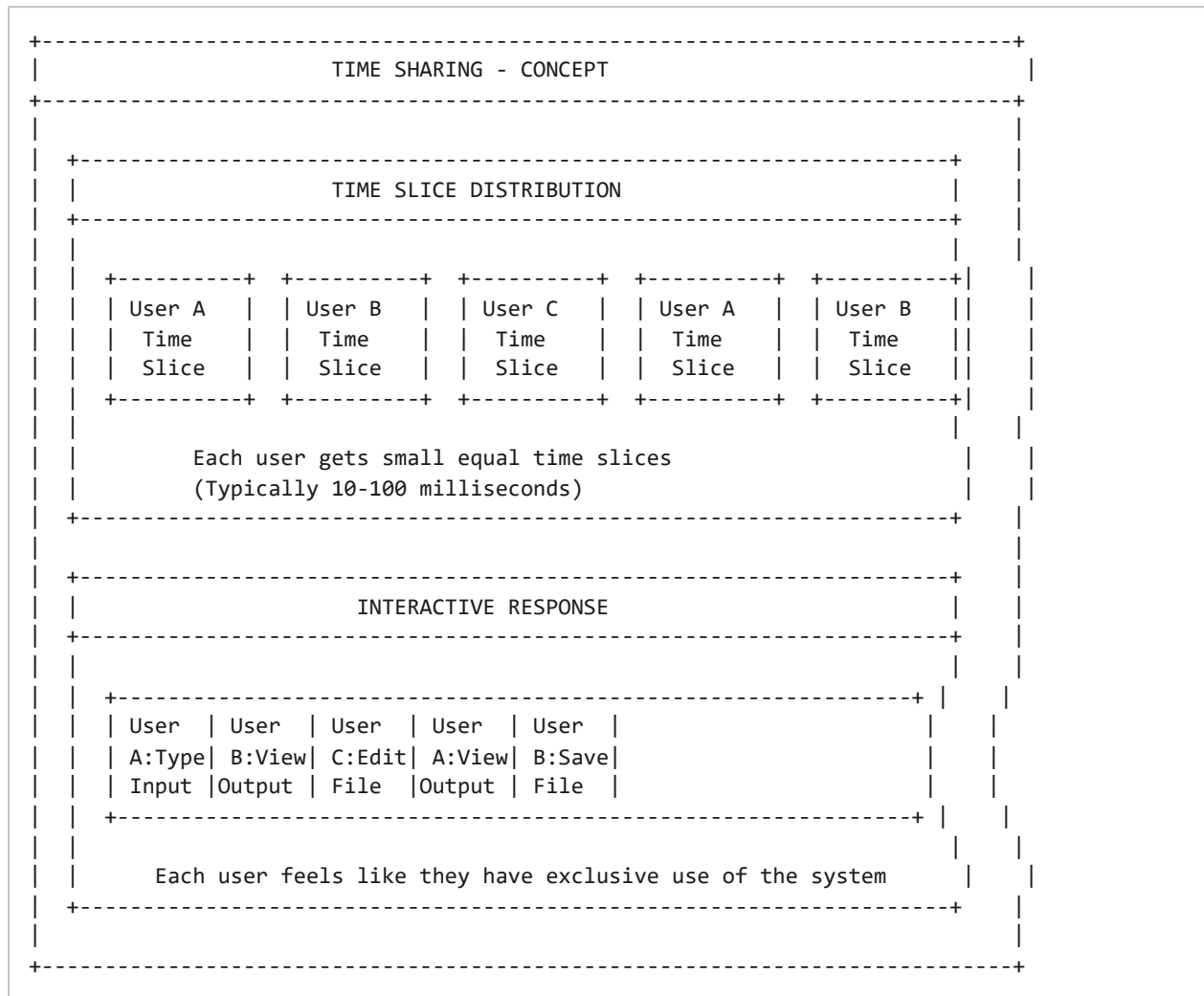
Complex scheduling algorithms

Memory management overhead

Better resource utilization	I/O and CPU requirements
Increased throughput	Priority inversion possible
User satisfaction	System overhead

1.3 Time Sharing

Time Sharing (also called **Multitasking**) is an extension of multi-programming where the CPU switches between programs so **quickly** that users feel they have **dedicated access** to the system. Each program gets a **time slice** (quantum) of CPU time.



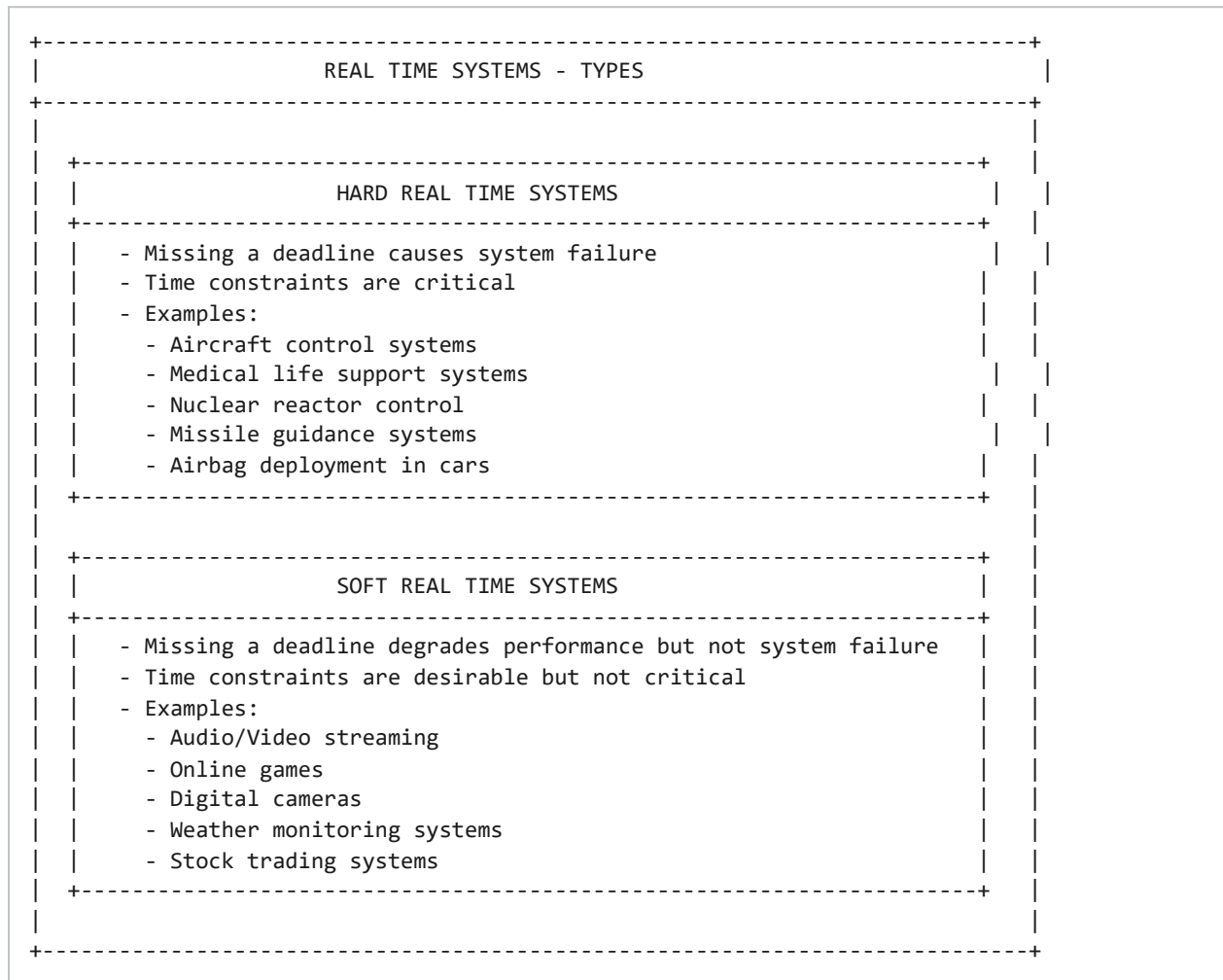
Time Sharing vs Multi-programming

Aspect	Multi-programming	Time Sharing
Primary Goal	Maximize CPU utilization	Minimize response time
Focus	Batch processing	Interactive processing
Context Switching	On I/O requests	On time slice expiry

User Interaction	Minimal	High
Response Time	Not critical	Critical
Typical Use	Mainframe batch jobs	Interactive terminals

1.4 Real Time Systems

A **Real Time System** is a system where the **correctness** of the system depends not only on the **logical correctness** of the computation but also on the **time** at which the result is produced. Results must be produced within specified **deadlines**.



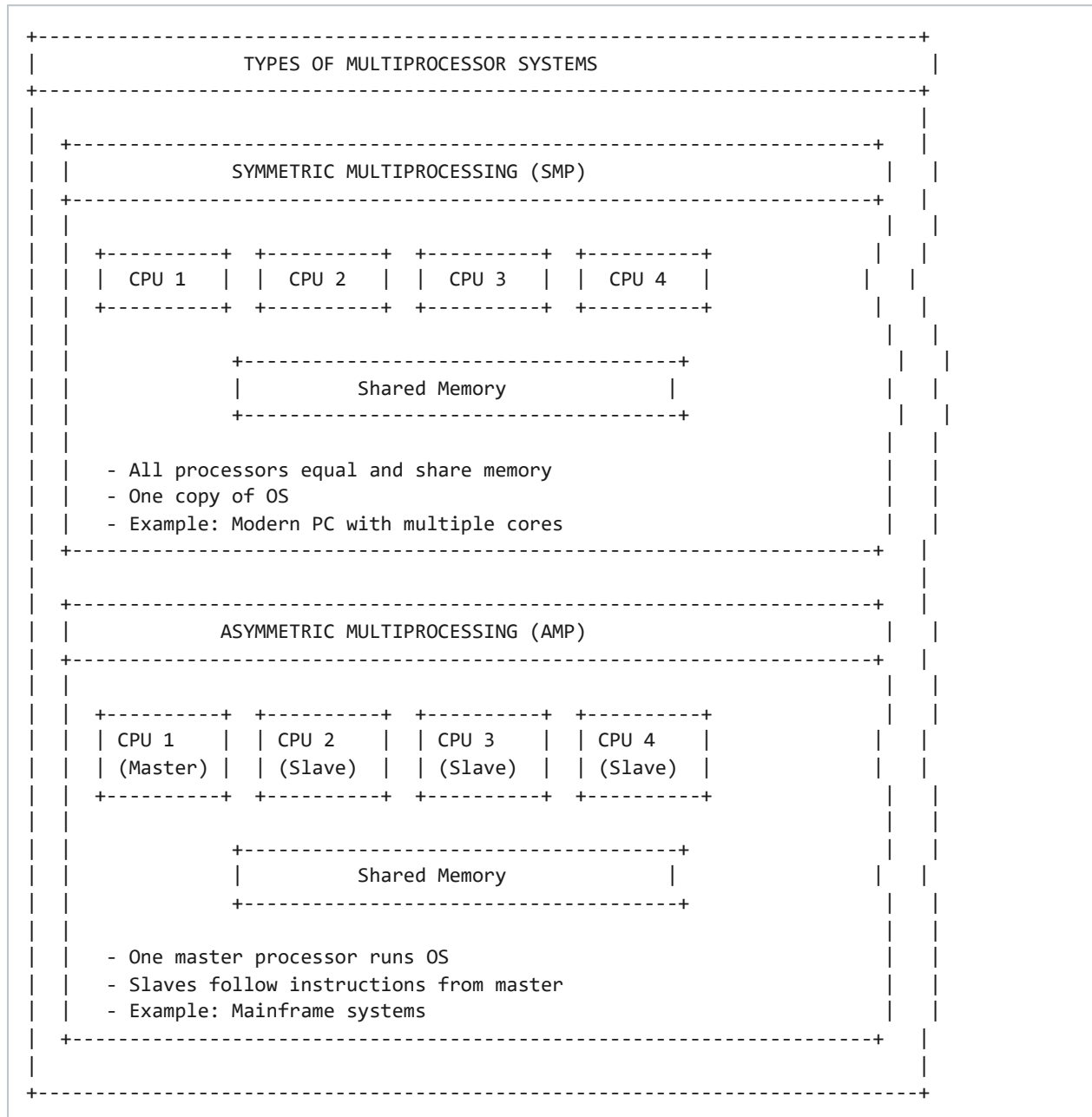
Characteristics of Real Time Systems

Characteristic	Description
Time Constraints	Must meet deadlines
Determinism	Predictable behavior
Responsiveness	Quick response to events
Safety	Must be reliable and safe

Concurrency	Handle multiple events simultaneously
Priority-based	Higher priority tasks execute first
Error Handling	Must handle errors gracefully
Stability	Must handle overload conditions

1.5 Multiprocessor Systems

A **Multiprocessor System** (also called **Parallel System**) is a system with **more than one CPU** (processor) sharing the **same memory** and **I/O devices**.



Advantages of Multiprocessor Systems

Advantage	Description
Increased Throughput	Multiple tasks executed simultaneously

Reliability	If one processor fails, others continue working
Graceful Degradation	System performance degrades gradually
Scalability	Add more processors to increase performance
Resource Sharing	Shared memory and I/O devices
Cost Efficiency	Better cost-performance ratio

Batch Processing is a technique where similar jobs are **grouped together** and executed **without user interaction**. Jobs are collected over time and processed in batches.

14/28

Simple Batch	Using monitor for automatic job sequencing
Multi-programmed Batch	Multiple jobs in memory simultaneously
Spooling Batch	Using disk as buffer for I/O

Advantages and Disadvantages

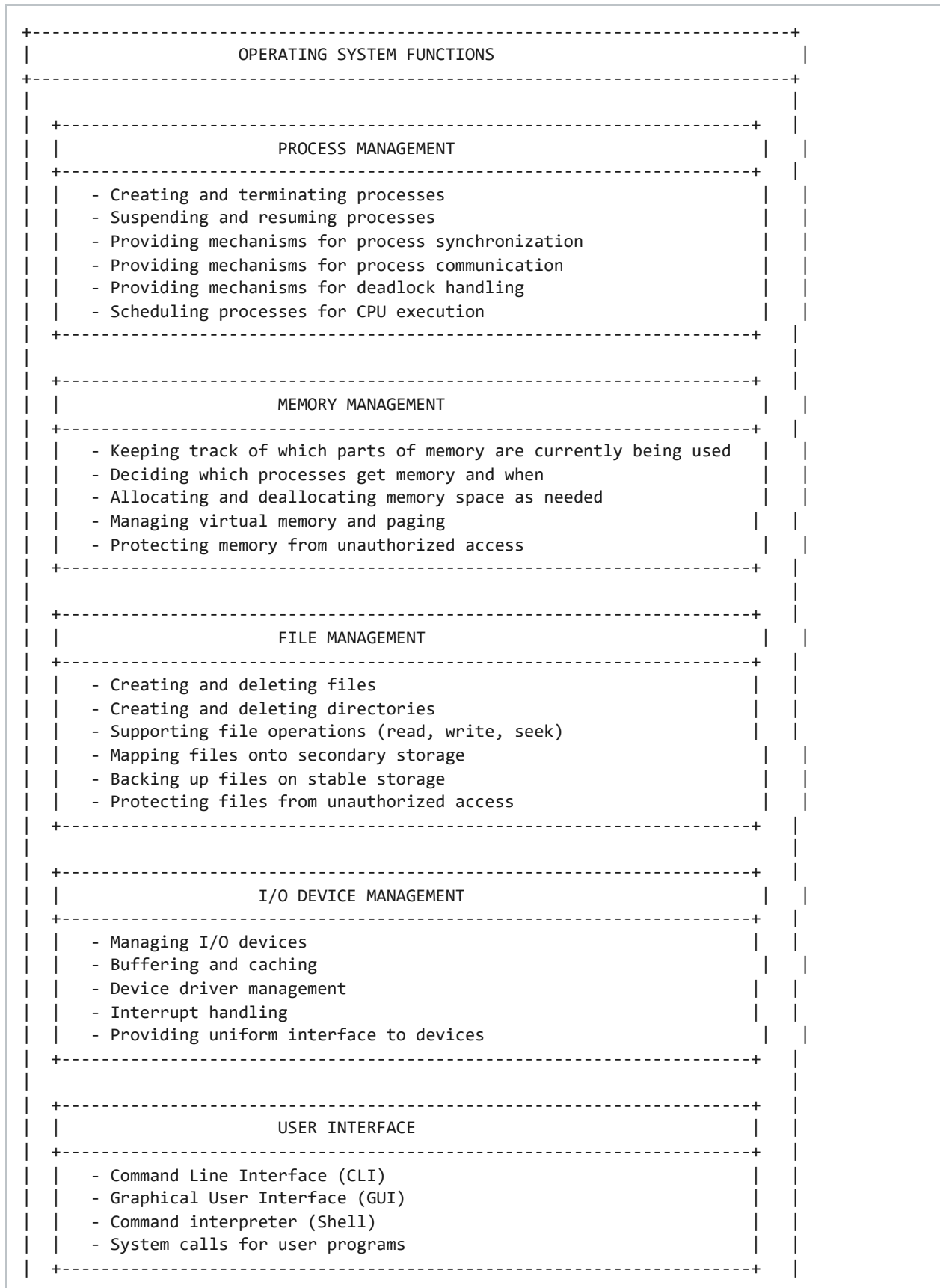
Advantages

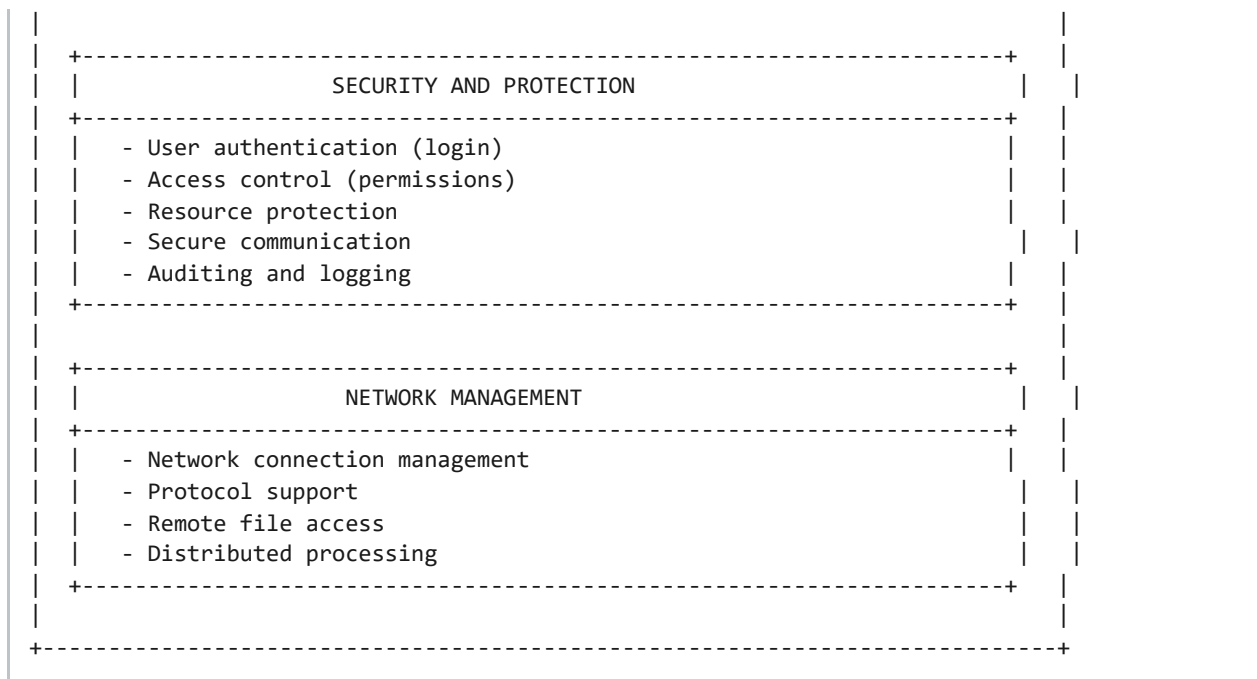
Disadvantages

Efficient for large volumes	No user interaction
Minimal resource contention	No priority jobs
Simple to implement	Long turnaround time
Reduced overhead	Wasted CPU during I/O
Best for regular jobs	Inflexible

Aspect	Single User	Multi-User
Number of Users	One at a time	Multiple simultaneously
Resource Sharing	Exclusive	Shared
Security	Minimal	Critical
Complexity	Simple	Complex
Cost	Low	High
Examples	PC, Laptop	Mainframe, Servers
User Accounts	None	Multiple accounts
Authentication	Not needed	Required

2. Functions of Operating System

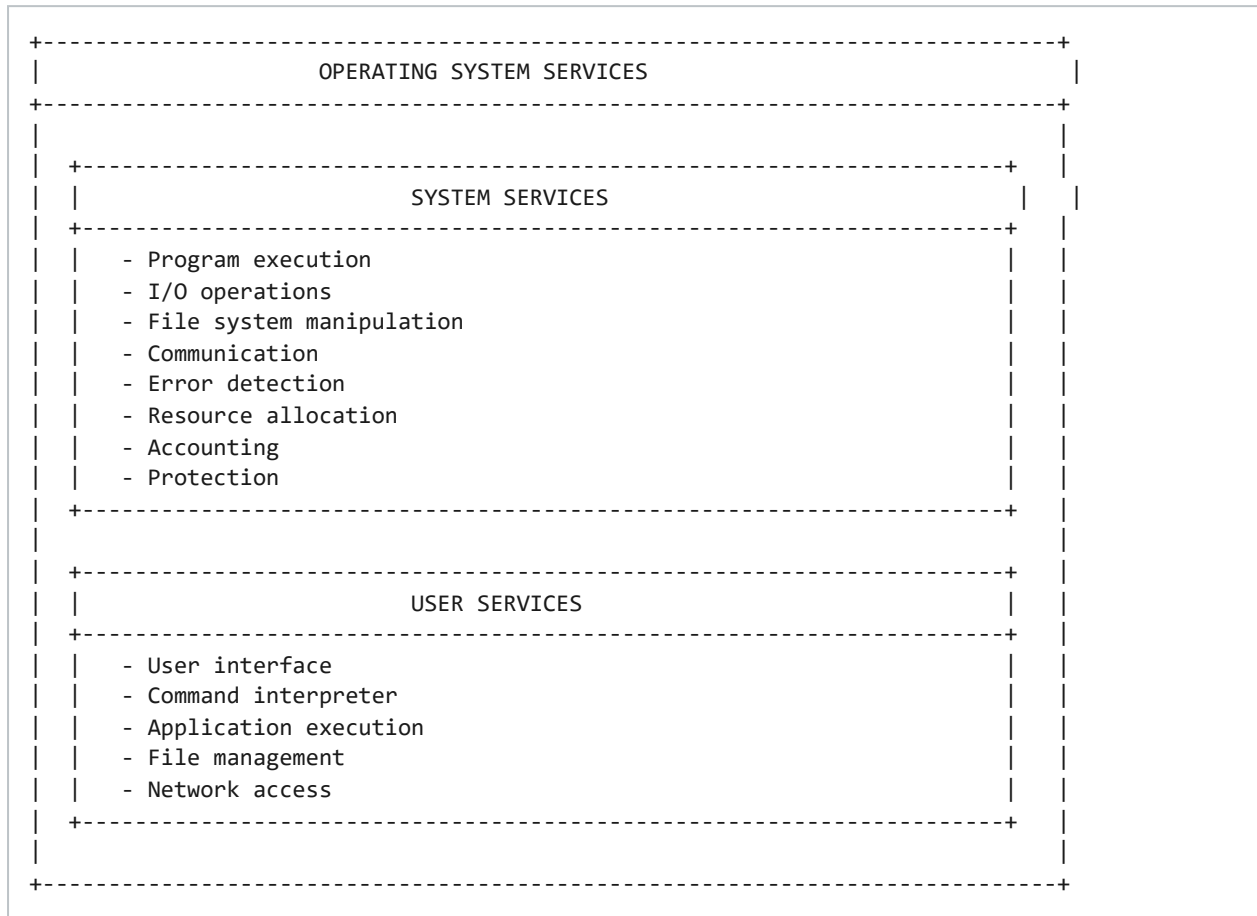




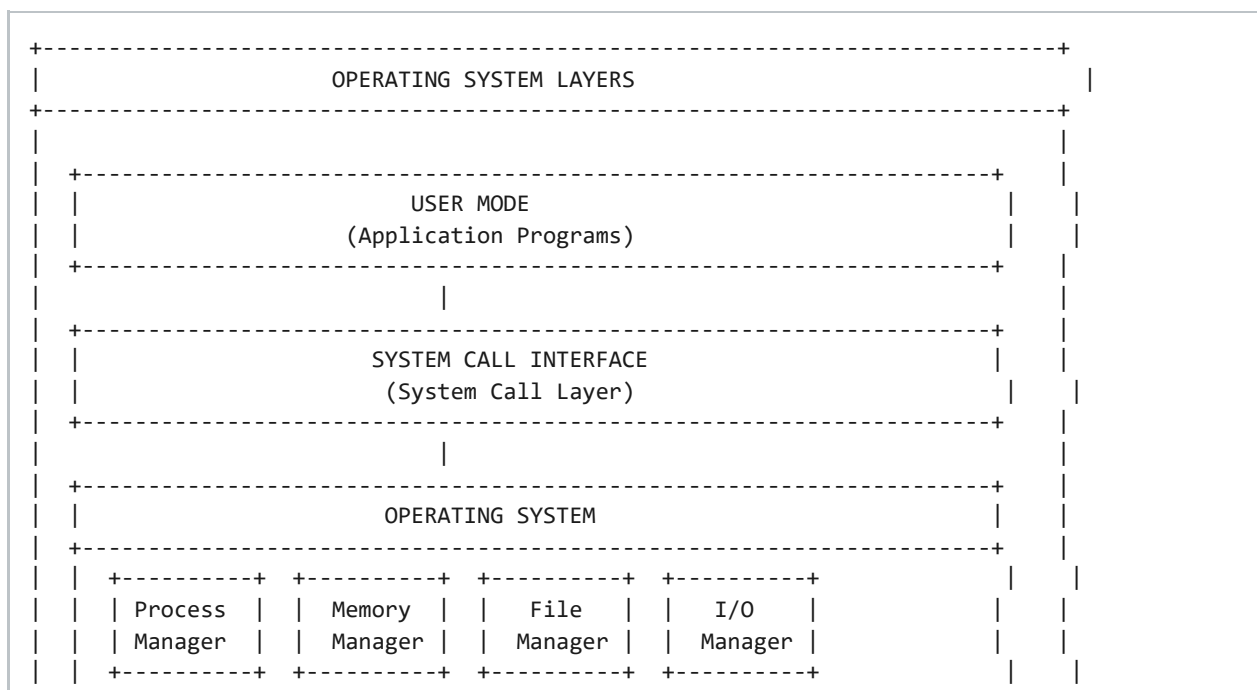
Characteristics of Operating System

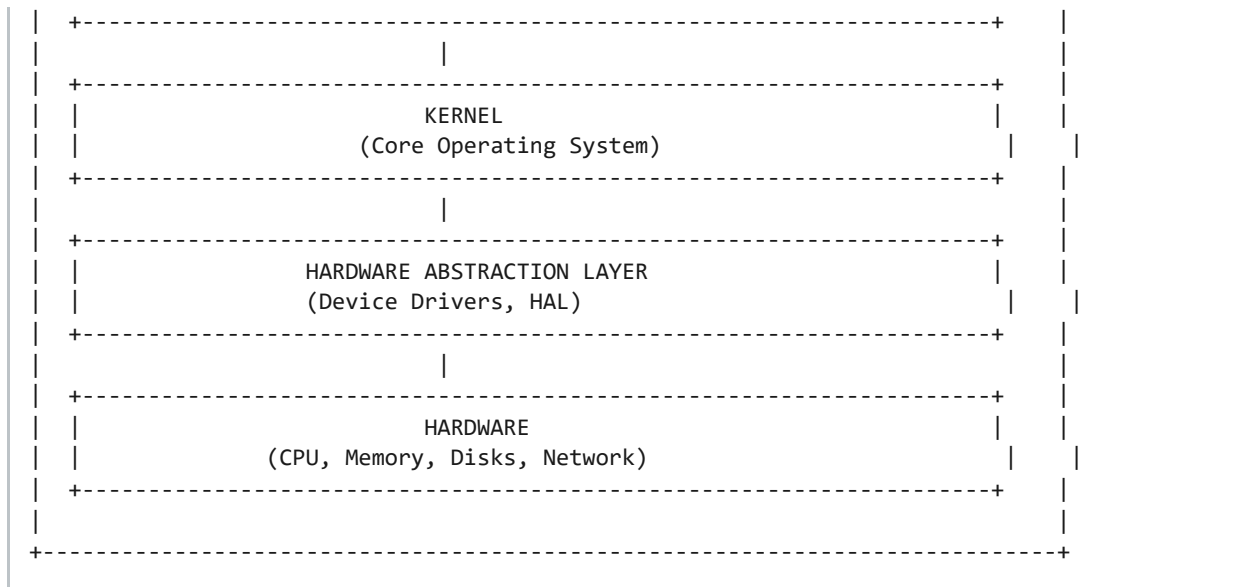
Characteristic	Description
Resource Management	Manages all hardware and software resources
Concurrency	Executes multiple tasks simultaneously
Security	Protects data and resources
Reliability	Operates continuously without failure
Efficiency	Maximizes resource utilization
Portability	Can run on different hardware platforms
Scalability	Can handle increasing workload
User-Friendly	Easy to interact with
Multitasking	Runs multiple programs concurrently
Real-Time Response	Responds to events in a timely manner

Operating System Services



Operating System Layers





3. Types of Operating Systems

3.1 Batch Processing

Batch Processing is a technique where similar jobs are **grouped together** and executed **without user interaction**. Jobs are collected over time and processed in batches.

Types of Batch Processing

Type	Description
Early Batch	Single job at a time, operator controlled
Simple Batch	Using monitor for automatic job sequencing
Multi-programmed Batch	Multiple jobs in memory simultaneously
Spooling Batch	Using disk as buffer for I/O

3.1 Multi-Programming

Allows multiple programs to be loaded into memory simultaneously. CPU switches between programs on I/O wait, keeping CPU busy at all times.

3.2 Time Sharing

Extension of multi-programming where CPU switches between programs so quickly that users feel they have dedicated access. Each program gets a time slice (quantum).

3.3 Real Time Systems

Correctness depends on both logical correctness and time. Results must be produced within deadlines.

3.4 Multiprocessor Systems

Systems with more than one CPU sharing the same memory and I/O devices, allowing parallel execution.

3.5 Distributed Systems

Collection of independent computers connected by a network, appearing as a single system to users.

3.6 Embedded Systems

Specialized computer systems designed for specific control functions within larger systems.

3.7 Mobile Operating Systems

Operating systems designed for mobile devices like smartphones and tablets (Android, iOS).

4. System Calls

A **System Call** is a mechanism through which a user program requests a service from the operating system kernel.

Types of System Calls

Type	Description	Examples
File Management	Operations on files	open(), read(), write(), close(), delete()
Process Management	Process creation and control	fork(), exec(), wait(), exit()
Device Management	I/O device operations	ioctl(), read(), write()
Information Management	Get/set system information	getpid(), time(), date()
Communication	Inter-process communication	pipe(), shmget(), msgget()

5. Operating System Structures

5.1 Simple/Monolithic Structure

The entire OS runs as a single program in kernel mode. All components are tightly integrated with direct function calls.

5.2 Layered Structure

OS is divided into layers, each built on top of the lower layer. Each layer provides services to the layer above.

5.3 Microkernel Structure

Minimal kernel providing only essential services (IPC, basic scheduling, memory management). Other services run in user space.

5.4 Modular Structure

OS is built using loadable kernel modules that can be loaded/unloaded at runtime.

5.5 Hybrid Structure

Combines elements of monolithic, microkernel, and modular approaches (e.g., Linux, Windows).

6. Kernel Types

6.1 Monolithic Kernel

All OS services run in kernel space. Fast but large and difficult to maintain. Examples: Linux, Unix.

6.2 Microkernel

Minimal kernel with most services in user space. More stable and secure but slower due to IPC overhead. Example: Minix, QNX.

6.3 Hybrid Kernel

Combines monolithic and microkernel approaches. Some services in kernel, some in user space. Example: Windows NT, macOS.

7. Booting Process

The **booting process** is the sequence of events that loads the operating system into memory when a computer is powered on.

Steps in Booting:

1. **Power On** - Power supply sends signal to motherboard
2. **BIOS/UEFI** - Basic Input/Output System performs POST (Power-On Self Test)
3. **Boot Loader** - BIOS loads boot loader from disk (MBR/GPT)
4. **OS Kernel** - Boot loader loads the OS kernel into memory
5. **Initialization** - Kernel initializes device drivers, memory management, and system processes
6. **User Login** - System presents login prompt or GUI

8. Quick Revision Points

Topic	Key Points
Operating System	Interface between user and hardware, manages resources
Simple Monitor	Earliest OS, executes one job at a time, no user interaction
Multi-Programming	Multiple programs in memory, CPU switches on I/O wait
Time Sharing	CPU time divided among users, interactive response
Real Time Systems	Must meet deadlines, Hard vs Soft real-time
Multiprocessor	Multiple CPUs, symmetric vs asymmetric
Batch Processing	Group jobs together, execute sequentially
Single User	One user at a time, PC/Laptop
Multi-User	Multiple users simultaneously, Mainframe/Server
Process Management	Create, schedule, terminate processes
Memory Management	Allocate, deallocate, protect memory
File Management	Create, delete, organize files
Device Management	Manage I/O devices, device drivers
Security	Authentication, access control, protection